```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.applications import vgg19

result_prefix = "image_generated"

# Weights of the different loss components
total_variation_weight = 1e-6
style_weight = 1e-6
content_weight = 2.5e-8
```

```
from PIL import Image
imgT = Image.open('/content/content.jpg')

imgB = Image.open('/content/style.jpg')


newsize = (400, 400)
imgB = imgB.resize(newsize)
imgB.show()
imgB = keras.preprocessing.image.img_to_array(imgB)
imgB = np.expand_dims(imgB, axis=0)
imgB = vgg19.preprocess_input(imgB)
style_reference_image = tf.convert_to_tensor(imgB)

imgT = imgT.resize(newsize)
imgT.show()
imgT = keras.preprocessing.image.img_to_array(imgT)
imgT = np.expand_dims(imgT, axis=0)
imgT = vgg19.preprocess_input(imgT)
base_image = tf.convert_to_tensor(imgT)

combination_image = tf.Variable(base_image)

img_nrows = 400
img_ncols = 400
```

```
Image.open('/content/content.jpg')
```

```
Image.open('/content/style.jpg')
```

```
def preprocess_image(image):
    # Util function to open, resize and format pictures into appropriate tensors
    img = keras.preprocessing.image.load_img(
        image, target_size=(img_nrows, img_ncols)
    )
    img = keras.preprocessing.image.img_to_array(img)
    img = np.expand_dims(img, axis=0)
    img = vgg19.preprocess_input(img)
    return tf.convert_to_tensor(img)
#
#
def deprocess_image(x):
    # Util function to convert a tensor into a valid image
    x = x.reshape((img_nrows, img_ncols, 3))
    # Remove zero-center by mean pixel
    x[:, :, 0] += 103.939
    x[:, :, 1] += 116.779
    x[:, :, 2] += 123.68
 #  'BGR'->'RGB'
    x = x[:, :, ::-1]
    x = np.clip(x, 0, 255).astype("uint8")
    return x
```

```
def gram_matrix(x):
    x = tf.transpose(x, (2, 0, 1))
    features = tf.reshape(x, (tf.shape(x)[0], -1))
    gram = tf.matmul(features, tf.transpose(features))
    return gram


# The "style loss" is designed to maintain
# the style of the reference image in the generated image.
# It is based on the gram matrices (which capture style) of
# feature maps from the style reference image
# and from the generated image


def style_loss(style, combination):
    S = gram_matrix(style)
    C = gram_matrix(combination)
    channels = 3
    size = img_nrows * img_ncols
    return tf.reduce_sum(tf.square(S - C)) / (4.0 * (channels**2) * (size**2))


# An auxiliary loss function
```

```
# designed to maintain the "content" of the
# base image in the generated image


def content_loss(base, combination):
    return tf.reduce_sum(tf.square(combination - base))


# The 3rd loss function, total variation loss,
# designed to keep the generated image locally coherent


def total_variation_loss(x):
    a = tf.square(
        x[:, : img_nrows - 1, : img_ncols - 1, :] - x[:, 1:, : img_ncols - 1, :]
    )
    b = tf.square(
        x[:, : img_nrows - 1, : img_ncols - 1, :] - x[:, : img_nrows - 1, 1:, :]
    )
    return tf.reduce_sum(tf.pow(a + b, 1.25))
```

```
# Build a VGG19 model loaded with pre-trained ImageNet weights
model = vgg19.VGG19(weights="imagenet", include_top=False)

# Get the symbolic outputs of each "key" layer (we gave them unique names).
outputs_dict = dict([(layer.name, layer.output) for layer in model.layers])

# Set up a model that returns the activation values for every layer in
# VGG19 (as a dict).
feature_extractor = keras.Model(inputs=model.inputs, outputs=outputs_dict)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg19/vgg19_weights_tf_dim_ordering_tf_ke
80134624/80134624 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
outputs_dict
```

```
{'input_layer': <KerasTensor shape=(None, None, None, 3), dtype=float32, sparse=False, name=keras_tensor>,
 'block1_conv1': <KerasTensor shape=(None, None, None, 64), dtype=float32, sparse=False, name=keras_tensor_1>,
 'block1_conv2': <KerasTensor shape=(None, None, None, 64), dtype=float32, sparse=False, name=keras_tensor_2>,
 'block1_pool': <KerasTensor shape=(None, None, None, 64), dtype=float32, sparse=False, name=keras_tensor_3>,
 'block2_conv1': <KerasTensor shape=(None, None, None, 128), dtype=float32, sparse=False, name=keras_tensor_4>,
 'block2_conv2': <KerasTensor shape=(None, None, None, 128), dtype=float32, sparse=False, name=keras_tensor_5>,
 'block2_pool': <KerasTensor shape=(None, None, None, 128), dtype=float32, sparse=False, name=keras_tensor_6>,
 'block3_conv1': <KerasTensor shape=(None, None, None, 256), dtype=float32, sparse=False, name=keras_tensor_7>,
 'block3_conv2': <KerasTensor shape=(None, None, None, 256), dtype=float32, sparse=False, name=keras_tensor_8>,
 'block3_conv3': <KerasTensor shape=(None, None, None, 256), dtype=float32, sparse=False, name=keras_tensor_9>,
 'block3_conv4': <KerasTensor shape=(None, None, None, 256), dtype=float32, sparse=False, name=keras_tensor_10>,
 'block3_pool': <KerasTensor shape=(None, None, None, 256), dtype=float32, sparse=False, name=keras_tensor_11>,
 'block4_conv1': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_12>,
 'block4_conv2': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_13>,
 'block4_conv3': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_14>,
 'block4_conv4': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_15>,
 'block4_pool': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_16>,
 'block5_conv1': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_17>,
 'block5_conv2': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_18>,
 'block5_conv3': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_19>,
 'block5_conv4': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_20>,
 'block5_pool': <KerasTensor shape=(None, None, None, 512), dtype=float32, sparse=False, name=keras_tensor_21>}
```

```
# List of layers to use for the style loss.
style_layer_names = [
    "block1_conv1",
    "block2_conv1",
    "block3_conv1",
    "block4_conv1",
    "block5_conv1",
]
# The layer to use for the content loss.
content_layer_name = "block5_conv2"


def compute_loss(combination_image, base_image, style_reference_image):
    input_tensor = tf.concat(
        [base_image, style_reference_image, combination_image], axis=0
    )
    features = feature_extractor(input_tensor)
    features
    # Initialize the loss
    loss = tf.zeros(shape=())

    # Add content loss
    layer_features = features[content_layer_name]
```

```
    base_image_features = layer_features[0, :, :, :]
    combination_features = layer_features[2, :, :, :]
    loss = loss + content_weight * content_loss(
        base_image_features, combination_features
    )
    # Add style loss
    for layer_name in style_layer_names:
        layer_features = features[layer_name]
        style_reference_features = layer_features[1, :, :, :]
        combination_features = layer_features[2, :, :, :]
        sl = style_loss(style_reference_features, combination_features)
        loss += (style_weight / len(style_layer_names)) * sl

    # Add total variation loss
    loss += total_variation_weight * total_variation_loss(combination_image)
    return loss
```

```
@tf.function
def compute_loss_and_grads(combination_image, base_image, style_reference_image):
    with tf.GradientTape() as tape:
        loss = compute_loss(combination_image, base_image, style_reference_image)
    grads = tape.gradient(loss, combination_image)
    return loss, grads
```

```
optimizer = keras.optimizers.SGD(
    keras.optimizers.schedules.ExponentialDecay(
        initial_learning_rate=100.0, decay_steps=100, decay_rate=0.96
    )
)


iterations = 5000
for i in range(1, iterations + 1):
    loss, grads = compute_loss_and_grads(
        combination_image, base_image, style_reference_image
    )
    optimizer.apply_gradients([(grads, combination_image)])
    if i % 100 == 0:
        print("Iteration %d: loss=%.2f" % (i, loss))
        img = deprocess_image(combination_image.numpy())
        fname = result_prefix + "_at_iteration_%d.png" % i
        keras.preprocessing.image.save_img(fname, img)
```

```
Iteration 100: loss=5653.28
Iteration 200: loss=4426.22
Iteration 300: loss=3912.64
Iteration 400: loss=3607.19
Iteration 500: loss=3396.57
Iteration 600: loss=3241.19
Iteration 700: loss=3119.10
Iteration 800: loss=3018.49
Iteration 900: loss=2934.33
Iteration 1000: loss=2863.31
Iteration 1100: loss=2802.22
Iteration 1200: loss=2748.74
Iteration 1300: loss=2701.57
Iteration 1400: loss=2659.66
Iteration 1500: loss=2622.17
Iteration 1600: loss=2588.50
Iteration 1700: loss=2558.05
Iteration 1800: loss=2530.25
Iteration 1900: loss=2504.79
Iteration 2000: loss=2481.53
Iteration 2100: loss=2460.17
Iteration 2200: loss=2440.45
Iteration 2300: loss=2422.17
Iteration 2400: loss=2405.19
Iteration 2500: loss=2389.44
Iteration 2600: loss=2374.76
Iteration 2700: loss=2361.05
Iteration 2800: loss=2348.25
Iteration 2900: loss=2336.24
Iteration 3000: loss=2325.03
Iteration 3100: loss=2314.54
Iteration 3200: loss=2304.72
Iteration 3300: loss=2295.47
Iteration 3400: loss=2286.77
Iteration 3500: loss=2278.58
Iteration 3600: loss=2270.88
Iteration 3700: loss=2263.62
Iteration 3800: loss=2256.74
Iteration 3900: loss=2250.24
Iteration 4000: loss=2244.08
Iteration 4100: loss=2238.25
```

```
Iteration 4200: loss=2232.73
Iteration 4300: loss=2227.49
Iteration 4400: loss=2222.52
Iteration 4500: loss=2217.80
Iteration 4600: loss=2213.31
Iteration 4700: loss=2209.03
Iteration 4800: loss=2204.96
Iteration 4900: loss=2201.08
Iteration 5000: loss=2197.40
```

```
Image.open('/content/image_generated_at_iteration_5000.png')
```